

Stacks

- a list of elements in which an element may be inserted or deleted only at the top of the stack
- the most recent element inserted is the first element to be removed
- elements are thus removed in the opposite order of that in which they were inserted.
- Also called **LIFO** (last-in-first-out) structure.

Examples

1. Stack of plates.
2. Stack of cards
3. Stack of books

Operations to be performed on a stack,

getHeight() – returns number of elements on stack

isEmpty() – returns true if stack is empty, else false is returned

Push(element) – places an element on the stack

Pop() – removes the top element from the stack

display() – displays the contents of the stack from the top most element downwards.

Implementation of a stack using an array

```
public class ArrayStack
{   private final int MAXSTK=100;

    private Object[] STACK = new Object[100];
    private int TOP;

    public ArrayStack()
    {   TOP = 0;
    }

    public int getHeight()
    {   return TOP;
    }
    public boolean isEmpty()
    {   return (TOP == 0);
    }

    public void Push(Object ITEM)
    {   if (TOP == MAXSTK)
        System.out.println("OVERFLOW - STACK is FULL ");
        else
        {   TOP = TOP + 1;
            STACK[TOP] = ITEM;
        }
    }

    public Object Pop()
    {   Object ITEM=null;
        if (TOP == 0)
            System.out.println("UNDERFLOW - STACK is EMPTY");
        else
        {   ITEM = STACK[TOP];
            TOP = TOP - 1;
        }
        return ITEM;
    }

    public void display()
    {   if (TOP != 0)
        {   for (int i=0; i<=TOP;i++)
            {   System.out.println("item "+i+" --> "+STACK[i]);
                System.out.println("END OF STACK");
            }
        }
        else
            System.out.println("Stack is empty - nothing to print");
    }
}
```